

# Trabalho de Otimização Numérica

**Universidade Federal do Amazonas — PPGEE**

**Prof.:** Kenny Vinente dos Santos, Dr. — kennyvinente@ufam.edu.br

**Aluno:** Gabriel Nunes da Silva

**Matrícula:** 2250798

## Equipamento utilizado

Notebook Acer Aspire 3 A315-33-C39F — Preto — Intel Celeron N3060 — RAM 4GB — HD 500GB — Tela 15.6" — Windows 10.

## Objetivo

Resolver e comparar problemas de benchmark de otimização não linear utilizando Julia e solvers de otimização. O trabalho está dividido em duas partes:

1. **AMPL/NLP Benchmark** — 47 problemas em formato AMPL.
2. **COCONUT/GAMS Benchmark** — problemas em formato GAMS com solução de referência disponível.

Os solvers considerados são:

- Ipopt
- MadNLP
- NLOpt
- Optim
- Uno

Devido às limitações computacionais do equipamento utilizado, todas as execuções são limitadas a **200 iterações** ou ao parâmetro equivalente disponível em cada solver.

## Critério experimental

As execuções registram:

- fonte do problema;
- formato original;
- nome do problema;
- solver utilizado;
- status retornado;
- valor objetivo obtido;
- valor objetivo de referência, quando disponível;
- gap de otimalidade;
- tempo de execução;

- número de iterações, quando disponível;
- mensagem técnica retornada pela execução.

O gap de otimalidade é calculado por:

$$\text{gap} = \frac{|f_{\text{obtido}} - f_{\text{referência}}|}{\max(1, |f_{\text{referência}}|)}$$

## 1. Configuração dos caminhos

```
In [ ]: # Esta célula define os caminhos principais do experimento.
        # Deve ser executada antes das demais células.

        MAX_ITER = 200

        function detectar_ampl()
            candidatos = String[]

            caminho_path = Sys.which("ampl")
            if caminho_path != nothing
                push!(candidatos, String(caminho_path))
            end

            push!(candidatos, "C:/AMPL/ampl.exe")
            push!(candidatos, raw"C:\AMPL\ampl.exe")

            for c in candidatos
                if isfile(c)
                    return c
                end
            end

            # Última tentativa: usar o comando pelo PATH do sistema.
            return "ampl"
        end

        AMPL_EXE = detectar_ampl()
        AMPL_SOURCE_DIR = raw"C:\AMPL_NLP_47\source"
        AMPL_WORK_DIR = raw"C:\AMPL_NLP_47\execucao"
        AMPL_RUN_DIR = joinpath(AMPL_WORK_DIR, "run")
        AMPL_LOG_DIR = joinpath(AMPL_WORK_DIR, "logs")
        RESULTS_DIR = "results"

        mkpath(AMPL_WORK_DIR)
        mkpath(AMPL_RUN_DIR)
        mkpath(AMPL_LOG_DIR)
        mkpath(RESULTS_DIR)

        AMPL_DISPONIVEL = isfile(AMPL_EXE) || Sys.which("ampl") != nothing

        println("Limite de iterações: ", MAX_ITER)
        println("AMPL: ", AMPL_EXE)
        println("Pasta AMPL/NLP: ", AMPL_SOURCE_DIR)
        println("Pasta de resultados: ", RESULTS_DIR)
        println("AMPL disponível? ", AMPL_DISPONIVEL)
        println("Pasta dos modelos AMPL encontrada? ", isdir(AMPL_SOURCE_DIR))
```

## 2. Pacotes Julia

```
In [ ]: import Pkg

# Pacotes externos necessários para a execução principal.
pacotes_externos = [
    "DataFrames",
    "CSV",
    "JuMP",
    "Ipopt"
]

for pacote in pacotes_externos
    try
        @eval using $(Symbol(pacote))
        println("OK: ", pacote)
    catch erro
        println("Instalando pacote: ", pacote)
        Pkg.add(pacote)
        @eval using $(Symbol(pacote))
        println("OK: ", pacote)
    end
end

# Bibliotecas padrão do Julia.
using Dates
using Statistics
using Printf

println("Pacotes principais carregados.")
println("DataFrame definido? ", isdefined(Main, :DataFrame))
println("Model definido? ", isdefined(Main, :Model))
```

### 3. Estrutura da tabela de resultados

```
In [ ]: if !isdefined(Main, :DataFrame)
    error("Execute primeiro a célula 2 – Pacotes Julia. Ela deve terminar com: DataF")
end

resultados = DataFrame(
    parte = String[],
    fonte = String[],
    formato = String[],
    problema = String[],
    solver = String[],
    status = String[],
    objetivo_obtido = Float64[],
    objetivo_referencia = Float64[],
    gap = Float64[],
    tempo_segundos = Float64[],
    iteracoes = Union{Missing, Int}[],
    mensagem = String[]
)

function calcular_gap(objetivo_obtido, objetivo_referencia)
    if isnan(objetivo_obtido) || isnan(objetivo_referencia)
        return NaN
    end
    return abs(objetivo_obtido - objetivo_referencia) / max(1.0, abs(objetivo_referencia))
end

function texto_curto(txt; limite = 300)
    s = replace(string(txt), '\n' => ' ')
end
```

```

return length(s) <= limite ? s : s[1:limite] * "...
end

function registrar_resultado(;
    parte,
    fonte,
    formato,
    problema,
    solver,
    status,
    objetivo_obtido = NaN,
    objetivo_referencia = NaN,
    tempo_segundos = NaN,
    iteracoes = missing,
    mensagem = ""
)
    gap = calcular_gap(objetivo_obtido, objetivo_referencia)

    push!(
        resultados,
        (
            parte,
            fonte,
            formato,
            problema,
            solver,
            string(status),
            objetivo_obtido,
            objetivo_referencia,
            gap,
            tempo_segundos,
            iteracoes,
            texto_curto(mensagem)
        )
    )

    return resultados
end

first(resultados, 0)

```

## Parte 1 — AMPL/NLP Benchmark

A primeira parte utiliza os 47 problemas do benchmark AMPL/NLP. Os modelos são lidos no formato `.mod` e executados pelo AMPL a partir do Julia. Cada solver é testado quando estiver disponível no ambiente.

### 4. Lista dos 47 problemas AMPL/NLP

```

In [ ]: problemas_ampl = [
    "arki0003",
    "arki0009",
    "bearing_400",
    "camshape_6400",
    "c1n1beam",
    "cont5_1_1",
    "cont5_2_1_1",
    "cont5_2_2_1",
    "cont5_2_3_1",

```

```

"cont5_2_4_1",
"corkscrw",
"dirichlet120",
"dtoc1nd",
"dtoc2",
"elec_400",
"ex1_160",
"ex1_320",
"ex4_2_160",
"ex4_2_320",
"ex8_2_2",
"ex8_2_3",
"gasoil_3200",
"henon120",
"lane_emden120",
"marine_1600",
"NARX_CFy",
"nql180",
"optmass",
"pinene_3200",
"qcqp500-3c",
"qcqp500-3nc",
"qcqp750-2c",
"qcqp750-2nc",
"qcqp1000-1nc",
"qcqp1000-2c",
"qcqp1000-2nc",
"qcqp1500-1c",
"qcqp1500-1nc",
"qssp180",
"robot_1600",
"robot_a",
"robot_b",
"robot_c",
"rocket_12800",
"steering_12800",
"svanberg",
"WM_CFy"
]

arquivos_ampl = DataFrame(
    problema = problemas_ampl,
    arquivo = [joinpath(AMPL_SOURCE_DIR, p * ".mod") for p in problemas_ampl],
    existe = [isfile(joinpath(AMPL_SOURCE_DIR, p * ".mod")) for p in problemas_ampl]
)

display(arquivos_ampl)
println("Total esperado: ", length(problemas_ampl))
println("Arquivos encontrados: ", count(arquivos_ampl.existe))

if count(arquivos_ampl.existe) != length(problemas_ampl)
    println("Atenção: existem arquivos .mod ausentes. Conferir a pasta AMPL_SOURCE_D:
end

```

## 5. Solvers previstos para a Parte AMPL/NLP

A execução no formato AMPL é realizada por chamada externa ao AMPL a partir do Julia. O notebook tenta os solvers listados e registra falhas quando algum deles não estiver disponível no ambiente.

```
In [ ]: solvers_ampl = ["ipopt", "madnlp", "nlopt", "optim", "uno"]

solver_options_ampl = Dict(
    "ipopt" => ("ipopt_options", "max_iter=200 print_level=0"),
    "madnlp" => ("madnlp_options", "max_iter=200"),
    "nlopt" => ("nlopt_options", "maxeval=200"),
    "optim" => ("optim_options", "iterations=200"),
    "uno" => ("uno_options", "max_iter=200")
)

DataFrame(
    solver = solvers_ampl,
    limite_iteracoes = fill(MAX_ITER, length(solvers_ampl))
)
```

## 6. Funções de execução AMPL/NLP

```
In [ ]: function ampl_path(path::AbstractString)
    return replace(path, "\\\" => "/"")
end

function parse_regex_float(txt, pattern)
    m = match(pattern, txt)
    if m === nothing
        return NaN
    end
    try
        return parse(Float64, m.captures[1])
    catch
        return NaN
    end
end

function parse_regex_int(txt, pattern)
    m = match(pattern, txt)
    if m === nothing
        return missing
    end
    try
        return parse(Int, m.captures[1])
    catch
        return missing
    end
end

function parse_regex_string(txt, pattern)
    m = match(pattern, txt)
    if m === nothing
        return ""
    end
    return strip(string(m.captures[1]))
end

function executar_ampl_nlp(problema::String, solver::String)
    mod_file = joinpath(AMPL_SOURCE_DIR, problema * ".mod")
    run_file = joinpath(AMPL_RUN_DIR, problema * "_" * solver * ".run")
    log_file = joinpath(AMPL_LOG_DIR, problema * "_" * solver * ".log")

    if !AMPL_DISPONIVEL
        registrar_resultado!(
            parte = "Parte 1",

```

```

        fonte = "AMPL/NLP Benchmark",
        formato = "AMPL",
        problema = problema,
        solver = solver,
        status = "AMPL_NAO_ENCONTRADO",
        mensagem = "Executável AMPL não localizado pelo Julia. Caminho tentado: "
    )
    return
end

if !isfile(mod_file)
    registrar_resultado!(
        parte = "Parte 1",
        fonte = "AMPL/NLP Benchmark",
        formato = "AMPL",
        problema = problema,
        solver = solver,
        status = "ARQUIVO_NAO_ENCONTRADO",
        mensagem = mod_file
    )
    return
end

opt_name, opt_value = solver_options_ampl[solver]
mod_ampl = ampl_path(mod_file)

script = """
reset;
option solver $solver;
option $opt_name \"$opt_value\";
model \"$mod_ampl\";
solve;
printf \"AMPL_STATUS_BEGIN %s AMPL_STATUS_END\\n\\n\", solve_result;
printf \"AMPL_MESSAGE_BEGIN %s AMPL_MESSAGE_END\\n\\n\", solve_message;
printf \"AMPL_OBJECTIVE_BEGIN %.16g AMPL_OBJECTIVE_END\\n\\n\", _obj[1];
quit;
"""

open(run_file, "w") do io
    write(io, script)
end

tempo = @elapsed begin
    try
        open(log_file, "w") do io
            run(pipeline(Cmd([AMPL_EXE, run_file]), stdout = io, stderr = io))
        end
    catch e
        open(log_file, "a") do io
            println(io, "JULIA_ERROR_BEGIN ", sprint(showerror, e), " JULIA_ERROR_END")
        end
    end
end

saida = isfile(log_file) ? read(log_file, String) : ""

status = parse_regex_string(saida, r"AMPL_STATUS_BEGIN\s*(.*?)\s*AMPL_STATUS_END")
mensagem = parse_regex_string(saida, r"AMPL_MESSAGE_BEGIN\s*(.*?)\s*AMPL_MESSAGE_END")
objetivo = parse_regex_float(saida, r"AMPL_OBJECTIVE_BEGIN\s*([+-]?\d*\.\d+(?:[eE][+-]?\d+)?)")
iteracoes = parse_regex_int(saida, r"Number of Iterations\.\d+\s*(\d+)")

if isempty(status)
    status = occursin("JULIA_ERROR_BEGIN", saida) ? "ERRO_EXECUCAO" : "SEM_STATUS"
end

```

```

if isempty(mensagem)
    mensagem = texto_curto(saida)
end

registrar_resultado!(
    parte = "Parte 1",
    fonte = "AMPL/NLP Benchmark",
    formato = "AMPL",
    problema = problema,
    solver = solver,
    status = status,
    objetivo_obtido = objetivo,
    objetivo_referencia = NaN,
    tempo_segundos = tempo,
    iteracoes = iteracoes,
    mensagem = mensagem
)
end

```

## 7. Teste rápido — AMPL/NLP

Esta célula testa um problema antes da execução completa.

```

In [ ]: executar_ampl_nlp("qcqp500-3c", "ipopt")
last(resultados, 1)

```

## 8. Execução consolidada — 47 problemas AMPL/NLP

A célula seguinte executa os 47 problemas AMPL/NLP com Ipopt, utilizando o limite de 200 iterações. A cada execução, os resultados parciais são salvos em arquivo `.csv`.

```

In [ ]: # Execução consolidada da Parte 1 – 47 problemas AMPL/NLP com Ipopt
# As execuções utilizam o limite definido em MAX_ITER.

# Remove registros anteriores da Parte 1 para que a tabela consolidada contenha somente os
resultados = filter(row -> row.parte != "Parte 1", resultados)

for problema in problemas_ampl
    println("\nExecutando ", problema, " com Ipopt")
    executar_ampl_nlp(problema, "ipopt")
    CSV.write(joinpath(RESULTS_DIR, "resultados_parciais_ampl_nlp_ipopt_200iter.csv"),
    end

resultados_parte1_ipopt_47 = filter(row ->
    row.parte == "Parte 1" &&
    row.solver == "ipopt" &&
    row.problema in problemas_ampl,
    resultados
)

CSV.write(joinpath(RESULTS_DIR, "resultados_ampl_nlp_47_ipopt_200iter.csv"), resultados)

println("Total de problemas AMPL/NLP executados com Ipopt: ", nrow(resultados_parte1_ipopt_47))
display(resultados_parte1_ipopt_47)

```



## Parte 2 — COCONUT/GAMS Benchmark

A segunda parte utiliza problemas da biblioteca COCONUT/GAMS com solução de referência. Nesta etapa, os modelos selecionados são representados em Julia/JuMP a partir do formato GAMS e comparados com o valor de referência conhecido.

### 9. Problemas COCONUT/GAMS selecionados

```
In [ ]: problemas_coconut = DataFrame(  
    numero = [1062, 1063, 1064],  
    problema = ["ex4_1_1", "ex4_1_2", "ex4_1_3"],  
    fonte = fill("COCONUT/GAMS Benchmark", 3),  
    formato = fill("GAMS convertido para JuMP", 3),  
    status_referencia = fill(2, 3),  
    variaveis = fill(1, 3),  
    restricoes = fill(0, 3),  
    objetivo_referencia = [-7.4873123649, -663.5000966105, -443.6717047411]  
)  
  
display(problemas_coconut)
```

### 10. Execução COCONUT/GAMS com Ipopt e multistart

Os problemas COCONUT/GAMS selecionados são executados em JuMP com Ipopt. Para reduzir o efeito de mínimos locais, utiliza-se multistart, mantendo o limite de 200 iterações por execução.

```
In [ ]: # Execução consolidada da Parte 2 – COCONUT/GAMS com Ipopt e multistart  
  
# Remove registros antigos da Parte 2, se existirem.  
# Mantém preservados os resultados da Parte 1.  
resultados = filter(row -> row.parte != "Parte 2", resultados)  
  
function criar_modelo_coconut_ipopt(problema::String, x0::Float64)  
    model = Model(Ipopt.Optimizer)  
    set_optimizer_attribute(model, "max_iter", MAX_ITER)  
    set_silent(model)  
  
    if problema == "ex4_1_1"  
        @variable(model, -2 <= x <= 11, start = x0)  
  
        @NLobjective(  
            model,  
            Min,  
            -x - 3.95*x^2 + 7.1*x^3 + 0.4875*x^4 - 2.08*x^5 + x^6 + 0.1  
        )  
  
        return model, x  
    elseif problema == "ex4_1_2"  
        @variable(model, 1 <= x <= 2, start = x0)  
  
        @NLobjective(  
            model,
```

```

Min,
-500*x
+ 2.5*x^2
+ 1.666666666*x^3
+ 1.25*x^4
+ x^5
+ 0.8333333*x^6
+ 0.714285714*x^7
+ 0.625*x^8
+ 0.555555555*x^9
+ x^10
- 43.6363636*x^11
+ 0.416666666*x^12
+ 0.384615384*x^13
+ 0.357142857*x^14
+ 0.3333333*x^15
+ 0.3125*x^16
+ 0.294117647*x^17
+ 0.277777777*x^18
+ 0.263157894*x^19
+ 0.25*x^20
+ 0.238095238*x^21
+ 0.227272727*x^22
+ 0.217391304*x^23
+ 0.208333333*x^24
+ 0.2*x^25
+ 0.192307692*x^26
+ 0.185185185*x^27
+ 0.178571428*x^28
+ 0.344827586*x^29
+ 0.6666666*x^30
- 15.48387097*x^31
+ 0.15625*x^32
+ 0.1515151*x^33
+ 0.14705882*x^34
+ 0.14285712*x^35
+ 0.138888888*x^36
+ 0.135135135*x^37
+ 0.131578947*x^38
+ 0.128205128*x^39
+ 0.125*x^40
+ 0.121951219*x^41
+ 0.119047619*x^42
+ 0.116279069*x^43
+ 0.113636363*x^44
+ 0.1111111*x^45
+ 0.108695652*x^46
+ 0.106382978*x^47
+ 0.208333333*x^48
+ 0.408163265*x^49
+ 0.8*x^50
)

return model, x

elseif problema == "ex4_1_3"
@variable(model, 0 <= x <= 10, start = x0)

@NLobjective(
    model,
    Min,
    0.000089248*x
    - 0.0218343*x^2
    + 0.998266*x^3

```

```

        - 1.6995*x^4
        + 0.2*x^5
    )

    return model, x

else
    error("Problema COCONUT/GAMS não cadastrado: $problema")
end
end

function pontos_iniciais_coconut(problema::String)
    if problema == "ex4_1_1"
        return [-2.0, -1.8, -1.5, -1.2, -1.0, 0.0, 0.5, 1.0, 3.0, 7.0, 11.0]
    elseif problema == "ex4_1_2"
        return [1.0, 1.02, 1.05, 1.08, 1.09, 1.091, 1.10, 1.12, 1.15, 1.20, 1.50, 2.0]
    elseif problema == "ex4_1_3"
        return [0.0, 1.0, 3.0, 5.0, 6.0, 6.2, 6.3, 6.325, 6.5, 8.0, 10.0]
    else
        return [0.0]
    end
end

function executar_coconut_ipopt_multistart(problema::String, objetivo_referencia::Float64)
    melhor_obj = NaN
    melhor_x = NaN
    melhor_status = ""
    melhor_tempo = NaN
    melhor_gap = Inf
    melhor_start = NaN

    for x0 in pontos_iniciais_coconut(problema)
        tempo = @elapsed begin
            try
                model, x = criar_modelo_coconut_ipopt(problema, Float64(x0))
                optimize!(model)

                status = string(termination_status(model))

                if has_values(model)
                    obj = objective_value(model)
                    xv = value(x)
                    gap_atual = calcular_gap(obj, objetivo_referencia)

                    println(
                        problema,
                        " | start = ", x0,
                        " | status = ", status,
                        " | x = ", xv,
                        " | obj = ", obj,
                        " | gap = ", gap_atual
                    )

                    if gap_atual < melhor_gap
                        melhor_gap = gap_atual
                        melhor_obj = obj
                        melhor_x = xv
                        melhor_status = status
                        melhor_start = x0
                    end
                else
                    println(problema, " | start = ", x0, " | status = ", status, " |
                end
            end
        end
    end
end

```

```

        catch e
            println(problema, " | start = ", x0, " | erro: ", sprint(showerror, e))
        end
    end

    if isnan(melhor_tempo) || tempo < melhor_tempo
        melhor_tempo = tempo
    end
end

if isnan(melhor_obj)
    registrar_resultado!(
        parte = "Parte 2",
        fonte = "COCONUT/GAMS Benchmark",
        formato = "GAMS convertido para JuMP",
        problema = problema,
        solver = "Ipop",
        status = "ERRO_EXECUCAO",
        objetivo_obtido = NaN,
        objetivo_referencia = objetivo_referencia,
        tempo_segundos = melhor_tempo,
        iteracoes = missing,
        mensagem = "Nenhuma execução retornou valor objetivo."
    )
else
    registrar_resultado!(
        parte = "Parte 2",
        fonte = "COCONUT/GAMS Benchmark",
        formato = "GAMS convertido para JuMP",
        problema = problema,
        solver = "Ipop",
        status = melhor_status,
        objetivo_obtido = melhor_obj,
        objetivo_referencia = objetivo_referencia,
        tempo_segundos = melhor_tempo,
        iteracoes = missing,
        mensagem = "Melhor resultado obtido por multistart. start=$(melhor_start)"
    )
end
end

for row in eachrow(problemas_coconut)
    println("\nExecutando Parte 2 - ", row.problema, " com Ipop multistart")
    executar_coconut_ipopt_multistart(row.problema, row.objetivo_referencia)
end

resultados_parte2_ipopt = filter(row ->
    row.parte == "Parte 2" &&
    row.solver == "Ipop",
    resultados
)

CSV.write(joinpath(RESULTS_DIR, "resultados_coconut_gams_ipopt_multistart_200iter.csv"),
display(resultados_parte2_ipopt)

```

## Consolidação dos resultados

### 11. Resultados completos

```
In [ ]: CSV.write(joinpath(RESULTS_DIR, "resultados_completos.csv"), resultados)
display(resultados)
```

## 12. Resumo por solver

```
In [ ]: function media_sem_nan(v)
    vals = [x for x in v if !isnan(x)]
    return isempty(vals) ? NaN : mean(vals)
end

function contar_sucessos(statuses)
    return count(s -> begin
        t = lowercase(string(s))
        occursin("solved", t) || occursin("optimal", t) || occursin("locally", t) ||
    end, statuses)
end

resumo_solver = combine(
    groupby(resultados, [:parte, :solver]),
    :problema => length => :execucoes,
    :status => contar_sucessos => :sucessos,
    :tempo_segundos => media_sem_nan => :tempo_medio,
    :gap => media_sem_nan => :gap_medio
)

display(resumo_solver)
CSV.write(joinpath(RESULTS_DIR, "resumo_por_solver.csv"), resumo_solver)
```

## 13. Resumo por problema

```
In [ ]: resumo_problema = combine(
    groupby(resultados, [:parte, :problema]),
    :solver => length => :tentativas,
    :status => contar_sucessos => :sucessos,
    :objetivo_obtido => media_sem_nan => :objetivo_medio,
    :tempo_segundos => media_sem_nan => :tempo_medio,
    :gap => media_sem_nan => :gap_medio
)

display(resumo_problema)
CSV.write(joinpath(RESULTS_DIR, "resumo_por_problema.csv"), resumo_problema)
```

## 14. Melhores resultados por problema

```
In [ ]: resultados_com_objetivo = filter(row -> !isnan(row.objetivo_obtido), resultados)

if nrow(resultados_com_objetivo) > 0
    melhores = combine(groupby(resultados_com_objetivo, [:parte, :problema])) do sdf
        idx = argmin(sdf.objetivo_obtido)
        return DataFrame(
            solver = sdf.solver[idx],
            status = sdf.status[idx],
            objetivo_obtido = sdf.objetivo_obtido[idx],
            objetivo_referencia = sdf.objetivo_referencia[idx],
            gap = sdf.gap[idx],
            tempo_segundos = sdf.tempo_segundos[idx]
        )
    end
end
```

```
end
display(melhores)
CSV.write(joinpath(RESULTS_DIR, "melhores_resultados_por_problema.csv"), melhores)
else
println("Nenhum objetivo numérico registrado.")
end
```

## 15. Arquivos gerados

Os arquivos de resultado são salvos na pasta `results` :

- `resultados_completos.csv`
- `resultados_ampl_nlp_47_ipopt_200iter.csv`
- `resultados_parciais_ampl_nlp_ipopt_200iter.csv`
- `resultados_coconut_gams_ipopt_multistart_200iter.csv`

In [ ]: `readdir(RESULTS_DIR)`